A5 · 学生实操手册 —— 打包双 jar + 局域网上线 +(可选)AI 图书助手 + 全流程复盘

> **对应**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元⑤(Part A 收官) **用法**:跟着一步一步做,每一步都有【操作】【预期产出】【✅ 怎么算做好】【卡住怎么办】。老师带做时你同步做,课后自己做也照这份来。 **前置**:前四单元的 C/S 图书馆已在你机器上跑通(建库 + 服务端 + 客户端 + 借还 + 管理端 + 监控窗口);本机装好 JDK 11、MySQL 8(`127.0.0.1:3306`,root / 12345678)。 **配套答案版**:`05-projects/cs-swing-library-demo/`(`build-jars.sh` / `README.md`;遇到卡点可对照)。

今天要做什么(先读这一段)

前四单元你把 C/S 图书馆一块一块搭了出来,**但代码只在你一台电脑上、只有你能跑**。今天做三件收官的事:

1. **打包**:把项目打成**两个可执行 jar**——服务端一个、客户端一个。别人不装开发环境就能跑。
2. **上线(C/S 版)**:一台电脑起服务端,**多台电脑起客户端、登录窗填服务器的局域网 IP** 连过来。开两个客户端**同时借最后一本**,验证服务端事务防超借。
3. **全流程复盘**:一张图串起这五个单元练的 Vibe Coding 流程。

> 💡 先记住一件事:我们这门课不是网页开发,所以"**上线**"不是"部署让浏览器访问",而是"**打成 jar + 局域网多台电脑连同一个服务端**"。这正好是 C/S 三段式的价值:数据库和业务只在服务端一台,客户端只管界面、不碰库。

> ⚠️ **动手前,先花点时间读下面两块"跟做前先读懂"**。前四单元你都在 IDE 里点"运行"就跑起来了——从没**自己打过包**、更没**让别的电脑连进来**过。"打成 jar"和"局域网多机"用到 jar / classpath / Main-Class、IP / 端口 / 防火墙这些新词。先把这两块读懂,后面跑 `build-jars.sh`、配局域网,你才知道每条命令在干嘛。

今天分步走(外加一个课后任务):

步骤	你要做的事	产出
Step 1	打包两个可执行 jar + 让 AI 逐行解释脚本	`dist/` 下 `library-server.jar` + `library-client.jar`,`java -jar` 能启动
Step 2	局域网多机演示 + 让 AI 帮你查 IP / 讲防火墙	别的电脑填服务器 IP 能连上;多客户端并发借最后一本只成一个
收官验收 A	**端到端回归测试:一条业务链路走到底 + 某站断了定位是哪一层**	一张十站测试用例表,逐条勾过;会顺着链子定位问题在哪一层
收官验收 B	**PRD 覆盖率自查:对照 PRD 打钩表自评 + 让 AI 反审缺口**	一张"做出来几成"的打钩表 + 必做项完成度% + 缺口补丁 prompt
Step 3	(可选/挑战)AI 图书助手	客户端聊天面板,接大模型查真实藏书回答(需自己的 key)
Step 4	Vibe Coding 全流程复盘	能把这五个单元复盘成一张图

> 💡 Step 1 / Step 2 / 收官验收 A / 收官验收 B 是**必做**;Step 3 是可选/挑战。**收官验收两步是今天上线前的"最后一道闸"**,别跳。

⚠️ 关于 Step 3(AI 助手)· 先说清楚

Step 3 是可选 / 挑战内容,需要**你自己的大模型 API Key**(DeepSeek,platform.deepseek.com 申请,通常有免费额度)。**没有 key 就跳过 Step 3,完全不影响今天的核心验收(打包双 jar + 局域网上线)。** 想做的同学课前把 key 准备好。key 是身份凭证,别发群里、别提交到公开仓库。

🧠 跟做前先读懂(一)· jar 打包零基础速记

打包前,先把几个新词认全(配生活类比,读一遍就好):

词	一句话	生活类比
jar	把一个项目编译出来的一堆 `.class` 打成的**一个文件**(后缀 `.jar`),谁拿到都能跑	一个 **zip 压缩包 / 安装包**(jar 本质就是 zip)
`.java` → `.class`	你写的 `.java`(源码)要先用 `javac` 编译成 `.class`(能跑的字节码),jar 里装的就是 `.class`	图纸(`.java`)→ 零件(`.class`)→ 装箱(`.jar`)
清单 Main-Class	jar 里一行配置,写明双击 / `java -jar` 时**从哪个类的 `main` 启动**(启动入口)	一栋楼的**大门**,告诉别人从哪进
classpath	告诉 Java"要用到的类 / jar 都在哪几个地方找",找不到就报 `ClassNotFoundException`	一张"**去哪几个书架找书**"的清单
fat jar	把依赖(这里是 MySQL 驱动)也**一起打进 jar**,拿到一个包就能连库,不用另外带驱动	把电池也装进玩具盒里,拆开就能玩
去签名	合并 MySQL 驱动时,要删掉它自带的签名文件,否则报签名错	撕掉原厂**防伪封条**,重新装箱(封条对不上新箱子会被安检拦)

为什么服务端要 fat jar、客户端不用:服务端要用 JDBC 连库,离不开 MySQL 驱动,所以把驱动打进去(fat jar);客户端不直连库、只发 Socket,不需要任何第三方依赖,普通 jar 就行。

最小命令示例(HelloWorld 级,认全三条命令):

```
javac -encoding UTF-8 -d out Hello.java      # ① 编译:.java → .class,放到 out/
jar cfe hello.jar Hello -C out .             # ② 打包:c 创建 / f 文件名 / e 入口类
java -jar hello.jar                          # ③ 运行:按清单 Main-Class 启动
```

> 记牢 `jar cfe` = **c**reate 创建 / **f**ile 文件名 / **e**ntry 入口类。图书馆的打包脚本再长,也就是在这三条上多做两件事:编译时挂上驱动、打服务端包时把驱动合并进去。

🧠 跟做前先读懂(二)· 局域网零基础速记

让别的电脑连进来,先认全这几个词(配"一栋楼"类比):

词	一句话	生活类比(一栋楼)
局域网	连在**同一个路由器 / WiFi 热点**下的一群机器,能互相直接找到、说话	同一**栋楼**,房间之间能串门、打内线
IP 地址	每台机器的门牌号,形如 `192.168.1.10`;`127.0.0.1`(localhost)= **我自己这台**	房间的**房间号**
端口	找到机器后,还要用端口号找到**具体哪个程序**;图书馆服务端用 **9000**,MySQL 用 3306	房间里的**电话分机号**
防火墙	每台电脑门口的"门卫",默认**拦掉外面主动打进来的连接**;要放行入站 9000 别人才连得进	楼下**门卫**,不放行访客就进不来

⚠️ **两个最容易踩的坑(今天连不上,九成是这俩)**:

1. **客户端登录窗还填 `127.0.0.1`** —— 那是"我自己这台",局域网下要填**服务器电脑的 IP**(如 `192.168.1.10`)。给隔壁房间打电话不能拨自己的内线。
2. **服务器电脑防火墙拦了 9000** —— IP 填对了还连不上,多半是门卫没放行。**服务器电脑要放行入站 TCP 9000**(Windows Defender 常默认拦,首次运行弹窗选"允许")。

为什么不用改代码就支持局域网:服务端那句 `new ServerSocket(9000)` 默认绑 `0.0.0.0`(本机所有网卡都听),本机连、局域网连它都接。所以从单机切局域网,**代码一行不改**,只需"客户端填对 IP + 服务器放行防火墙"。

查一台电脑的局域网 IP:

```
# Windows:命令行 ipconfig,找"无线局域网适配器"下的 IPv4 地址(如 192.168.1.10)
ipconfig
# Mac / Linux:
ifconfig | grep 192          # 或  ipconfig getifaddr en0
```

Step 1 · 打包两个可执行 jar

我们的系统本来就是两个独立运行的程序,所以打**两个 jar**:

jar	是什么	谁运行	含 MySQL 驱动吗
`library-server.jar`	服务端(监听 9000 + 多线程 + 连库)	服务器电脑(一台)	**要**(服务端要连数据库)→ 打成 **fat jar**
`library-client.jar`	客户端(纯 Swing 界面)	客户端电脑(多台)	**不要**(客户端不直连库)

> 💡 两个词先记住(详见上面"跟做前先读懂一"):

> - **fat jar** = 把 MySQL 驱动一起打进 jar。**服务端要连库,所以打 fat jar。**
> - **去签名** = MySQL 官方驱动 jar 自带签名文件(`META-INF/` 下的 `*.SF` / `*.RSA` / `*.DSA`)。合并进自己的 jar 后签名对不上,一跑就报签名错。**所以合并时要删掉这些签名文件。** 这是打 fat jar 最经典的坑。

步骤 1.1 · 让 Trae 生成打包脚本

【操作】 在 Trae SOLO 对话框里,**原样复制**下面这段(取自抓手2 单元⑤ Step 1)输入并回车:

```
教我把这个 C/S 项目打成两个可执行 jar:
- 客户端 jar(纯 Swing,Main-Class=client.ui.LoginFrame);
- 服务端 fat jar(把 MySQL 驱动合并进去、去掉驱动签名,Main-Class=服务端监控窗口
server.ui.ServerMonitorFrame);
给我一个 build-jars.sh:先 javac 全部源码,再分别 jar cfe;告诉我怎么 java -jar 启动。
```

【预期产出】 Trae 会生成一个 `build-jars.sh`。它的核心是三步,对照答案版应该长这样:

```
# ① 编译:全部源码编译到 out/(编译时把驱动放进 classpath)
javac -encoding UTF-8 -cp "lib/mysql-connector-j-8.0.33.jar" -d out
@/tmp/_srcs.txt

# ② 客户端 jar:纯 Swing,入口登录窗,不含驱动
jar cfe dist/library-client.jar client.ui.LoginFrame -C out .

# ③ 服务端 fat jar:合并驱动 → 删签名 → 打包,入口是监控窗口
cp -r out/* stage/
( cd stage && unzip -oq "../lib/mysql-connector-j-8.0.33.jar" )
rm -f stage/META-INF/*.SF stage/META-INF/*.RSA stage/META-INF/*.DSA stage/META-
INF/MANIFEST.MF
rm -rf stage/META-INF/versions
jar cfe dist/library-server.jar server.ui.ServerMonitorFrame -C stage .
```

> 说明:`jar cfe` 里 `c`=创建、`f`=文件名、`e`=入口类(Main-Class)。所以服务端 jar 双击进监控台,客户端 jar 双击进登录窗。第 ③ 步那两行 `rm` 就是"去签名";`unzip 驱动.jar` 就是把驱动当 zip 解开(jar 本质是 zip)。

【✅ 怎么算做好】

- 你拿到一个 `build-jars.sh`,内容里有:编译带 `-encoding UTF-8` + `-cp` 挂驱动、客户端入口 `client.ui.LoginFrame`、服务端合并驱动 + 删 `META-INF` 签名 + 入口 `server.ui.ServerMonitorFrame`。

【卡住怎么办】

- **AI 忘了删签名**:回它一句"打服务端 fat jar 前,把解开的驱动里 `META-INF/*.SF`、`*.RSA`、`*.DSA`、`MANIFEST.MF` 和 `META-INF/versions` 删掉再打包。只改打包这一步。"
- **AI 把两个 jar 合成一个**:提醒"要分成两个 jar,客户端不含驱动、服务端才是 fat jar"。

步骤 1.2 · ⭐ 让 AI 逐行解释脚本(把速记对上号)

【操作】 脚本生成后,别急着跑——先让 AI 当老师,把它逐行讲一遍。SOLO 输入:

```
用中文逐行解释这个 build-jars.sh:每一条命令在做什么。特别讲清楚:
javac 的 -cp 为什么要带那个 MySQL 驱动;jar cfe 的 c/f/e 各是什么意思;
第 ③ 步的 unzip 和那两行 rm 在干嘛(为什么合并驱动要删 META-INF 里的签名文件)。
```

【✅ 怎么算做好】

- AI 讲得能和"跟做前先读懂一"对上号:`-cp` = 编译时把驱动放进 classpath(否则 `import` 驱动报找不到类);`jar cfe` = 创建 / 文件名 / 入口类;`unzip` = 把驱动 jar 当 zip 解开;两行 `rm` = 删签名文件(否则合并后签名对不上、跑起来报签名错)。**哪句对不上号,回上面速记再看一眼。**

> 💡 这一步别跳过。答辩老师问"你这个服务端 jar 怎么打的、为什么删签名",你能答上来,靠的就是这一遍。

步骤 1.3 · 跑脚本 + 启动

【操作】 (命令与答案版 README 一致)

```
# 打包(答案版 dist/ 已带现成 jar;想自己重打就跑这句)
./build-jars.sh

# 1) 先起服务端(fat jar,已内置驱动;会弹出【服务端监控台】窗口)
java -jar dist/library-server.jar

# 2) 再起客户端(纯 Swing,可开多个)
java -jar dist/library-client.jar
#    可选:指定服务端地址   java -jar dist/library-client.jar 127.0.0.1 9000
```

> ⚠️ 启动服务端前,先确认这台电脑上 MySQL 已建好库(前四单元的 `init.sql` 跑过)、MySQL 服务在运行。

【✅ 怎么算做好(Step 1 的验收点)】

- `dist/` 下出现两个文件:`library-server.jar`(约 **2.5MB**,含驱动)、`library-client.jar`(约 **77KB**,纯 Swing)——一大一小,大的那个带了驱动(就是速记里说的"胖包 / 瘦包")。
- `java -jar dist/library-server.jar` 弹出**服务端监控台窗口**(能看到监听端口 9000、运行状态、在线客户端数、实时日志)。
- `java -jar dist/library-client.jar` 弹出登录窗;用 `admin/123456` 能登录进去。

【卡住怎么办】

现象	可能原因	怎么办
服务端 jar 一跑就报 `SecurityException` / `... digest error`(签名错)	合并驱动**没删签名**	让 AI 在打包前删掉 `META-INF/*.SF`、`*.RSA`、`*.DSA`、`MANIFEST.MF`、`META-INF/versions` 再打(见 1.1)
服务端报 `No suitable driver` / `ClassNotFoundException`(找不到驱动)	驱动没合并进去,不是 fat jar	确认第 ③ 步有"解开驱动到 stage、再一起打包"
双击 jar **没反应**,或报 `no main manifest attribute`	入口类(Main-Class)没指对	服务端入口 `server.ui.ServerMonitorFrame`,客户端入口 `client.ui.LoginFrame`
中文乱码	编译没带 UTF-8	`javac` 要带 `-encoding UTF-8`(答案版脚本已带)
服务端启动报连不上数据库	MySQL 没起 / 没建库	先确认这台 MySQL 在跑、`init.sql` 跑过

Step 2 · 局域网多机演示

> 最好和邻座配合、两台电脑做。没有第二台也行:先在**同一台**上把"服务器地址填 127.0.0.1"这套跑通,理解"填 IP"的逻辑,考试 / 答辩时换成真 IP 即可。

服务端的 `ServerSocket` 默认绑 `0.0.0.0`(本机所有网卡),**天生支持局域网访问,不用改代码**(详见"跟做前先读懂二")。要让别的电脑连上,只需做对四件事:

```
[客户端电脑 A] ┐
[客户端电脑 B] ┼─ 同一 WiFi/路由器 ──→ [服务器电脑:MySQL + library-server.jar]
[客户端电脑 C] ┘      登录窗填服务器 IP + 端口 9000      防火墙放行入站 TCP 9000
```

步骤 2.1 · 让 Trae 讲清局域网怎么配

【操作】 在 Trae SOLO 里**原样复制**下面这段(取自抓手2 单元⑤ Step 2)输入并回车:

```
教我局域网多台电脑演示:一台电脑起服务端(MySQL + 服务端 jar),其它电脑起客户端,
登录窗"服务器地址"填服务端电脑的局域网 IP。告诉我服务端电脑要注意什么。
```

步骤 2.2 · ⭐ 让 AI 帮你查 IP + 教你放行防火墙

【操作】 做服务器的那台电脑,SOLO 输入(把系统换成你实际的 Windows 或 Mac):

```
我这台是 Windows(或 Mac),要当图书馆服务端。教我:① 用什么命令查我这台电脑的局域网 IP,
IPv4 那一行怎么认;② 怎么放行入站 TCP 9000(Windows 防火墙 / Mac 防火墙),
让局域网别的电脑能连进来。给我具体步骤。
```

【✅ 怎么算做好】

- 你照 AI 说的查出了自己的局域网 IP(形如 `192.168.x.x`),并把入站 9000 放行了(或首次运行弹窗选了"允许")。把这个 IP 报给做客户端的同学。

步骤 2.3 · 照着配四件事(与答案版 README「⑥ 单机 or 局域网」一致)

【操作】

1. **服务器电脑(一台)**:建库(`mysql -uroot -p12345678 < sql/init.sql`)+ 起服务端(`java -jar dist/library-server.jar`)。**MySQL 和服务端只在这一台。**
2. **查这台电脑的局域网 IP**(步骤 2.2 已查):Windows `ipconfig` 找 IPv4;Mac `ifconfig | grep 192`。
3. **客户端电脑(其它几台)**:起 `java -jar dist/library-client.jar`,登录窗**服务器地址填服务器电脑的 IP**(如 `192.168.1.10`,**不是** `127.0.0.1`)、端口填 `9000`。
4. **服务器电脑防火墙放行入站 TCP 9000**(步骤 2.2 已放行)。所有电脑连**同一个 WiFi / 路由器**。

> 💡 登录窗上有一行提示:"同一台电脑填 127.0.0.1;局域网填服务器电脑的 IP(如 192.168.1.10)"。填错这个,就是连不上最常见的原因(两大坑之一)。

【✅ 怎么算做好】

- 客户端电脑填对服务器 IP + 端口 9000,点登录能连上;用 `zhangsan/123456` 登录进读者端、能查书借书。**多台客户端能同时在线操作同一个服务端**(服务端监控台上在线客户端数会变多)。

【卡住怎么办】

现象	可能原因	怎么办
客户端**连不上**服务器	登录窗还填着 `127.0.0.1`(坑一)	服务器地址要填**服务器电脑的 IP**,不是 `127.0.0.1`(那是客户端自己)
客户端连不上,IP 也填对了	服务器电脑**防火墙拦了 9000**(坑二)	临时放行 / 允许弹窗;或手动加入站规则放行 TCP 9000
能连上但**登录失败**	服务器上没建库 / 没种子账号	确认服务器电脑 `init.sql` 跑过,有 `admin/zhangsan/lisi`
手机 / 别的电脑完全连不到	不在同一网,或 WiFi 客户端隔离	确认连的是**同一个 WiFi**;公共 WiFi 换成手机热点试

步骤 2.4 · ⭐ 并发防超借验证(务必做一次,答辩会问)

> 这是 C/S 上线最有说服力的一幕:**多台客户端同时借同一本只剩最后一本的书,看会不会超借。** 这个演示单机时代凑不齐多台客户端,今天局域网上线才做得成。

【操作】(照答案版 README「七、并发防超借测试步骤」)

1. 把某本书库存改成 1(答案版《万历十五年》初始 `stock=1`,可直接用它)。
2. 开 2~3 个客户端,分别用 `zhangsan` / `lisi` 登录。
3. **同时**对同一本书点"借这本书"。

【✅ 怎么算做好(Step 2 的验收点)】

- **只有一个客户端提示"借阅成功",其余提示"库存不足,借阅失败"**——库存不会被扣成负数,**不会超借**。这说明服务端的事务 + `WHERE stock>0` 按影响行数判定生效了。

【卡住怎么办】

- **两个都提示成功(超借了)**:说明服务端 `borrow` 没在一个事务里、或没用 `UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0` 按影响行数判。回单元③的审查点,用**纠错三步法**让 AI 补:"借书要在一个事务里减库存(`WHERE stock>0` 按影响行数)+ 写记录,任一失败回滚。只改借书这一处。"

★ 收官验收 A · 端到端回归测试 —— 一条业务链路走到底(必做)

> 打好包、局域网也连通了,但这只是"单点都对"。上线前最后要问一句:**从注册一个新读者开始,一条真实业务链路能不能一路走到底、每一站都对?** 这就是**端到端回归测试**。前四单元新加的注册 / 审核 / 续借 / 罚款 / 统计 / 审计,各自测过,但**串起来跑通过吗?** 今天上线这一刻,把它们连成一条链子拉一遍。

我们模拟一个真读者的完整旅程,一共**十站**,一站都不能断:

```
注册 → 管理员审核通过 → 登录 → 查书 → 借书 → 续借 → 还书(制造逾期→罚款)
     → 管理端统计报表看到这笔 → 审计日志查到这些操作        (+ 并发借最后一本只成一个)
```

步骤 A.1 · 让 AI 生成一份端到端测试用例表

【操作】 在 Trae SOLO 里**原样复制**下面这段输入并回车:

```
我要给这个 C/S 图书馆做一次端到端回归测试(局域网上线后验收)。请帮我生成一份"端到端测试用例表",
覆盖这条完整业务链路(一站都不要漏):
新读者注册 → 管理员审核通过 → 该读者登录 → 查书 → 借一本书 → 续借 → 还书(制造逾期,产生罚款)
→ 切管理端看借阅统计报表里有这笔 → 操作审计日志里查得到上面这些操作。
每条用例给我:序号、操作步骤、在哪一端做(读者端/管理端)、发出的协议命令(如 REGISTER|.. /
USER_APPROVE|.. /
LOGIN|.. / BOOK_QUERY|.. / BORROW|.. / RENEW|.. / RETURN|..)、预期结果、一个"是否通过"的勾选框 □。
再单列一组"负向用例":① 审核通过前就登录 → 应被拒;② 两个客户端并发借最后一本 → 只成一个;
③ 续借次数超上限 → 应被拒。用中文表格输出,我要照着一条条勾。
```

【预期产出】 一张十来行的用例表,骨架大致长这样(命令名以你项目实际协议为准):

```
| # | 操作步骤                | 端      | 协议命令                      | 预期结果                                                  | □ |
|---|-----------------------|--------|--------------------------|---------------------------------------------------------|---|
| 1 | 注册新读者 test01       | 读者端 | REGISTER|test01|123456|..| 提示"注册成功,待审核",status=pending | □ |
| 2 | 管理员审核通过 test01 | 管理端 | USER_APPROVE|test01       | test01 状态 pending→active          | □ |
| 3 | test01 登录           | 读者端 | LOGIN|test01|123456       | 登录成功,进读者端主界面             | □ |
| 4 | 查书                  | 读者端 | BOOK_QUERY|Java|0         | 列表含《Java核心技术》,显示可借数     | □ |
| 5 | 借这本书              | 读者端 | BORROW|<bookId>           | 借阅成功,库存-1,t_borrow 新增一条   | □ |
| 6 | 续借                  | 读者端 | RENEW|<borrowId>          | 续借成功,应还日期后延,续借次数+1     | □ |
| 7 | 还书(已制造逾期)      | 读者端 | RETURN|<borrowId>         | 归还成功,按逾期天数×0.5 写 t_fine   | □ |
| 8 | 管理端看统计报表      | 管理端 | STATS_BORROW_BY_CATEGORY | 该分类借阅数+1;罚款统计含这笔        | □ |
| 9 | 管理端查审计日志      | 管理端 | LOG_LIST                  | 查到 test01 注册/借/还 + 管理员审核 | □ |
```

> ⚠️ 表里的命令名(`REGISTER` / `USER_APPROVE` / `STATS_*` / `LOG_LIST` 等)以**你项目实际用的协议为准**——AI 生成后要和你自己的 `Api` / 协议文档核一遍,不一致就让它改成你的真实命令名。

怎么"制造逾期"不用等真到期:借完后,在服务器电脑上手动把这条借阅的应还日期改早,再去还书,就会算出逾期罚款——

```
UPDATE t_borrow SET due_time = DATE_SUB(CURDATE(), INTERVAL 1 DAY)
WHERE user_id = (SELECT id FROM t_user WHERE username='test01') AND status='借出';
```

步骤 A.2 · 照着用例表,一站一站勾过去

【操作】 两台电脑就绪(接 Step 2 的局域网环境),照下面的清单一条条做、一条条勾。**断在哪一站就停下来记下来**,先别急着改。

【✅ 怎么算做好(收官验收 A 的验收点)】

```
[端到端回归 · 十站验收]
□ 站1  注册 test01 后,库里 status=pending(SELECT status FROM t_user WHERE
username='test01')
□ 站2  管理端审核通过后,status 变 active
□ 站3  审核前登录被拒 / 审核后登录成功(负向 + 正向都验)
□ 站4  查书能查到、可借数正确
□ 站5  借书成功且库存-1、t_borrow 多一条
□ 站6  续借成功、应还日期后延;再续一次超上限被拒
□ 站7  还书后 t_fine 有一条,金额 = 逾期天数×0.5(自己手算核对)
□ 站8  管理端统计报表这笔进了聚合数字(借阅数 / 罚款额都对)
□ 站9  审计日志能查到 test01 这一串 + 管理员那次审核
□ 并发 两客户端同借最后一本,只成一个(接 Step 2 那一幕)
```

步骤 A.3 · ⭐ 某一站断了,怎么定位是哪一层(回归测试最练人的地方)

端到端最大的价值,不只是"发现坏了",而是**顺着链子快速指出坏在哪一层**。对照这张表:

断在哪一站(现象)	最可能是哪一层	怎么确认	可直接复制的追问 prompt
客户端连不上、注册都发不出去	**网络层**	IP 填错 / 防火墙拦 9000 / 服务端没起	`客户端连不上服务端,帮我按"IP 是否填服务器电脑的、防火墙 9000 是否放行、服务端 jar 是否在跑"三条逐一排查,给我每条确认命令。`
审核后仍登录不了	**认证/数据层**	`SELECT status` 看是不是真 active	`test01 审核通过了但登录还是失败,帮我查 USER_APPROVE 有没有真把 status 改成 active、登录校验是不是只放行 active。定位后只改这一处。`
查书为空	**数据层**	库建了没 / 有没有数据	`查书返回空,帮我确认 t_book 有数据且 BOOK_QUERY 模糊匹配 SQL 写对了。`
借书失败但库存明明有	**业务/事务层**	回单元③:事务 + `WHERE stock>0` 影响行数	`借书总失败但库存>0,帮我检查 borrow 是不是在一个事务里、有没有用 WHERE stock>0 按影响行数判。只改借书这一处。`
还了书但没罚款 / 金额不对	**业务规则层**	逾期天数×0.5、有没有写 t_fine	`还书没写罚款/金额不对,帮我核对逾期天数怎么算、单价是不是 0.5、有没有 INSERT 进 t_fine。`
统计报表看不到这笔	**聚合SQL层**	先确认数据真写了,再看 GROUP BY	`刚借的这笔统计里没有,帮我先确认数据真写进去了,再检查统计 GROUP BY 有没有算进它。`
审计日志查不到	**日志埋点层**	操作有没有写 t_operation_log	`test01 的借/还审计日志查不到,帮我确认这些操作成功后有没有写一条 t_operation_log。`

> 💡 记住这条对应关系:**连不上=网络层、登录不了=认证层、借书不扣=业务层、统计没数=聚合层、日志没记=埋点层。** 答辩官问"哪里出了问题你怎么查",你顺着这张表答,就是"懂系统"。

步骤 A.4 · ⭐ 让用例表升级:能定位 + 主动练故障 + 测完清场

【操作】 在**同一个对话**里继续追加(别开新对话),SOLO 输入:

```
在刚才的测试用例表基础上补三件事,别删原来的用例:
① 每条"预期结果"后面加一列"这一步若失败,最可能是哪一层(网络/认证/数据/业务/聚合SQL/日志埋点)",方便我断了秒定位;
② 教我"主动制造故障再定位"的练法:分别故意(a)把客户端 IP 填错、(b)关掉服务器 MySQL、(c)不审核就登录、(d)借库存为 0 的书,
   告诉我每种会在客户端看到什么现象、去哪一层查、一句修复方向;
③ 加一条"回滚清理"用例:测完把 test01 这个测试读者连同它的借阅/罚款/日志记录删掉,别污染演示数据,给我具体 SQL。
```

【✅ 怎么算做好】

- 拿到升级版用例表:每条带"失败=哪一层"、有一组"主动制造故障→现象→哪一层→修复方向"的练习、还有一条"测完清场"的 SQL。**故意把 IP 填错跑一次,亲眼看它断在网络层**——你就真会"定位"了。

> 🔬 这一步用到两招:**主动制造 bug 再定位**(故意搞坏再练找)、**让 AI 解释"现象↔哪一层"**。会"故意搞坏再精准定位",比"碰巧没坏"值钱得多。

> ⏸️ **做完的打个 1**。十站全绿的最好;卡在某一站的,先**说出你判断它断在哪一层**,再动手改——这一步练的就是"定位"。

★ 收官验收 B · PRD 覆盖率自查 —— 对照图纸打钩(必做)

> 回归测试验的是"跑通那条链子对不对";覆盖率自查换个角度:**当初 PRD(`.trae/rules` 那张图纸)上写要做的功能,我到底做出来几成?** 这是 Part A 收官的一次"对着图纸盘点",呼应今天的复盘——**PRD 就是那把尺子,规范是新的源代码。**

步骤 B.1 · 让 AI 对照 PRD 生成"做出来了吗"打钩表

【操作】 先把你的 `.trae/rules` / PRD 打开或附给 Trae,SOLO 输入:

```
这是我这个 C/S 图书馆项目当初的 PRD(命令 / 窗体 / 业务规则,见 .trae/rules)。
请帮我生成一张"PRD 功能 → 我做出来了吗"自查打钩表,分三大类列全:
① 协议命令(LOGIN / REGISTER / BOOK_QUERY / BORROW / RETURN / RENEW / MY_BORROW / USER_APPROVE /
   统计类 STATS_* / 审计 LOG_LIST / 分类 CRUD ... 按我 PRD 实际写的列);
② 窗体界面(登录窗 / 注册窗 / 读者端查借还 / 管理端图书CRUD / 用户审核 / 统计面板 / 服务端监控窗 ...);
③ 业务规则(借书事务防超借 / 续借上限 / 逾期罚款计算 / 用户审核三态 / 管理员与读者权限区分 ...)。
每行给我:功能项、属于"必做"还是"加分项"、我的完成度勾选框(□未做 □半成 □完成 三选一)、一句验收标准。
用中文表格,我要照着自评。
```

【预期产出】 一张三段(命令 / 窗体 / 规则)的对照表,每行带"必做/加分 + 完成度勾选框 + 验收标准"。示例:

```
| 功能项                | 必做/加分 | 完成度                | 验收标准
   |
|-------------------|----------|--------------------|-------------------------------
----|
| 借书事务防超借        | 必做      | □未做 □半成 ☑完成 | 两客户端并发借最后一本只成一个       |
| 逾期罚款计算          | 必做      | □未做 ☑半成 □完成 | 还书按逾期天数×0.5 写 t_fine
|
| AI 图书助手           | 加分      | ☑未做 □半成 □完成 | 问书能查真实藏书回答、不编造
|
```

步骤 B.2 · 老老实实按实际情况勾

【操作】 对着刚跑完的回归测试结果,一行一行如实勾"未做 / 半成 / 完成"。**别全标完成糊弄自己**——"半成"的行,写一句"还差什么"(如:罚款只算了天数没写进库)。

【✅ 怎么算做好】

```
[PRD 覆盖率自查]
□ 三大类(命令 / 窗体 / 规则)都列全了,没漏 PRD 上写过的功能
□ 每一行都按"未做/半成/完成"真实勾了
□ "半成"的行写了一句"还差什么"
□ 分清了必做项 vs 加分项(AI 助手 / 分页 / 高级统计属加分)
```

【卡住怎么办】

现象	怎么办(可直接复制的追问 prompt)
AI 把加分项也算进必做,完成度被拉低吓人	`请只按 PRD 里的"必做"功能算完成度,加分项(AI助手/分页/高级统计)单独列一栏,别混进必做。`
AI 报喜不报忧,一堆功能替我标"完成"	`别只看代码在不在。对每个你标"完成"的功能,给我一条能当场验的命令/操作,我实测不过的请重新标"半成"。`
表太笼统,"借还"揉成一行看不出缺口	`把"借还"拆成 借书 / 还书 / 续借 / 逾期罚款 四条分别打钩,粒度细到我能一条条验。`

步骤 B.3 · ⭐ 让 AI 当审查员,挑缺口 + 给补丁 prompt + 算完成度

【操作】 填好表后,在**同一对话**里继续,SOLO 输入:

```
我把上面的打钩表按实际情况填好了(见我标注的未做/半成/完成)。请你当审查员帮我两件事:
① 从我标"未做 / 半成"的里挑出:哪些是必做项(不补会影响验收 / 答辩)、哪些是加分项(锦上添花),按优先级排序;
② 每个缺口给我一句"补它的最小 prompt",我能直接拿去让 AI 补;
③ 给我一个百分比:必做项完成度大概多少。别报喜不报忧,缺的直说。
```

【✅ 怎么算做好(收官验收 B 的验收点)】

- 拿到:一份**按优先级排好的缺口清单**、每个缺口一句**能直接用的补丁 prompt**、一个**必做项完成度百分比**。到 80% 以上说明这套图书馆基本立起来了;不到也别慌——你手里正好有"缺口 + 补法",课后照着补。

> 🔬 这一步同时练三条方法论:**规范优先**(拿 PRD 当尺子量成品)、**信任但验证**(不靠"感觉做完了",一条条验、还让 AI 反审你的自评)、**产品优先于项目**(分清必做 vs 加分,先保能用能验收)。**知道自己差在哪,本身就是本事。**

> ⏸️ 把你的**必做项完成度百分比**打在公屏上,低于 80% 的我们一起看差哪。

Step 3 ·(可选 / 挑战)AI 图书助手

> ⚠️ **可选 / 挑战,需要你自己的大模型 key**。没 key 直接跳到 Step 4,不影响今天的验收。

给客户端加一个聊天面板,读者用大白话问书,AI **去查真实藏书**再回答。背后的机制叫 **Function Calling**:你告诉大模型"我有一个工具 `searchBooks` 能查图书馆的书",大模型遇到查书问题时**决定调用它**,你的程序查库、把真实结果给它,它再组织成人话。

```
① 读者问:"推荐一本 Java 的书"
        ↓  客户端把问题 + 工具说明(searchBooks)发给大模型
② 大模型判断:要查库 → 回"请调用 searchBooks(keyword='Java')"
        ↓  客户端执行:发 BOOK_QUERY|Java|0 给服务端 → 服务端查 t_book → 返回真实藏书
③ 查到 → 把结果发回大模型
        ↓
④ 大模型组织成人话回答 → 显示在聊天面板
```

> 💡 注意:AI 助手加在**客户端**,但查库这一步仍走 Socket 发 `BOOK_QUERY` 给服务端(由服务端查库)。**客户端依然不碰数据库,档 B 三段式没被破坏。** 大模型负责"理解 + 说话",你的系统负责"查真实数据"。

步骤 3.1 · 准备 key

【操作】

- 去 **platform.deepseek.com** 注册、创建一个 API Key(形如 `sk-xxxx`),复制保存好(通常只显示一次)。
- 把 key 填进客户端 AI 助手要读取的配置处(生成代码后 Trae 会告诉你填哪)。

【✅ 怎么算做好】

- key 已就绪。base-url 用 `https://api.deepseek.com/v1`(注意结尾 `/v1`),模型 `deepseek-chat`。

步骤 3.2 · 让 Trae 生成 AI 助手

【操作】 在 Trae SOLO 里**原样复制**下面这段(取自抓手2 单元⑤ Step 3)输入并回车:

```
给客户端加一个"AI 图书助手"聊天面板:接大模型(用我自己的 key),
读者问"推荐一本 Java 的书 / 有没有《活着》",AI 通过 Function Calling 调服务端的
BOOK_QUERY 查库,
再用自然语言回答。定义一个 searchBooks 工具,两轮对话(第一轮带工具让模型决定调不调,
第二轮把查库结果喂回让它作答)。key 没配置时友好提示,不要崩溃。
```

【预期产出(Function Calling 两轮的骨架)】

```
第一轮:带上 searchBooks 工具调大模型  → 大模型返回 tool_calls(它决定要查库)
        ↓ 解析出 keyword,客户端发 BOOK_QUERY|keyword|0 给服务端,服务端查 t_book 返回真实
藏书
第二轮:把"大模型的 tool_calls + 查库结果"追加进对话,再调一次大模型 → 拿到自然语言回答
        ↓ 显示在聊天面板
```

- 关键:给大模型的 **system 提示词要强调"只能根据 searchBooks 查到的真实藏书回答,不许编造书名"**——防止它一本正经胡说八道。

【✅ 怎么算做好】

- 配好 key 后,聊天面板问"推荐一本 Java 的书",回答里的书是**真从数据库查出来的**(去库里核对确实有);问一本库里没有的书,助手如实说"没找到",不瞎编。
- **没配 key** 时,助手给**友好提示**(如"未配置大模型 key"),而不是崩溃报错。

【卡住怎么办】

现象	怎么办
返回"未配置 API Key"	key 没填 / 没生效 → 填好 key 后重启客户端
报 401 / Unauthorized	key 错了或额度用完 → 检查有没有复制全 / 多空格,去平台确认有效
请求一直转圈	到大模型网络不通,或 base-url 写错 → 确认能上外网、`base-url` 是 `https://api.deepseek.com/v1`
回答的书**库里没有**(在编)	Function Calling 没走通 → 查第一轮有没有带 `searchBooks` 工具、第二轮有没有把查库结果喂回、system 有没有"不许编造"
没 key	直接跳过这一步,不影响验收

Step 4 · Vibe Coding 全流程复盘

这五个单元,你从一句"做个图书馆系统",一步步做出了一套能登录、能借还、有管理端和监控台、能局域网多机并发使用、还能打包给别人跑的**完整 C/S 系统**。回头看,你走的是这样一条流程:

```
反向提问理需求 → 写 Agent PRD(.trae/rules 图纸)
   → SOLO 分步生成(建库 → 服务端 → 登录链路 → 客户端 → 借还 → 管理端 → 监控窗口)
      → C/S 联调 → 纠错三步法 → 代码审查(信任但验证)→ 打包双 jar + 局域网上线
```

你真正学到的,不是"敲代码",而是这四件更值钱的事:

1. **把需求说清楚**——反向提问 + 写 Agent PRD,让 AI 知道到底要做什么(规范是新的源代码)。
2. **指挥 AI 分步干活**——一个功能一句话 +"只做 X,别动其他",让 AI 稳定产出。
3. **判断 AI 写得对不对**——信任但验证。尤其借书事务、每客户端一线程这些关键点,你得会盯着 AI。
4. **让系统跑起来给人用**——联调 + 打包 + 局域网上线,走完最后一公里。

> 🔬 这套"说清需求 + 指挥 AI + 判断对错 + 上线"的能力,就是 AI 时代最稀缺的东西。你学的不是敲得快,是**想得清、说得明、审得动**。

今天做完了什么(自检清单)

- [ ] 读懂了两块速记:jar / javac / Main-Class / classpath / fat jar / 去签名;局域网 / IP / 端口 / 防火墙 / 0.0.0.0
- [ ] 打出两个 jar:`library-server.jar`(fat jar,含驱动、去签名)+ `library-client.jar`(纯 Swing),`java -jar` 能启动(Step 1)
- [ ] 让 AI 逐行解释了 `build-jars.sh`,每条命令都对得上速记(Step 1.2)
- [ ] 局域网多机连通:别的电脑填服务器 IP + 9000 能登录、能查书借书;防火墙放行了 9000(Step 2)
- [ ] 并发防超借验证过:多客户端同时借最后一本,只成一个(Step 2)
- [ ] 端到端回归:注册→审核→登录→查→借→续→还→逾期罚款→统计报表→审计日志,一条链子十站勾过;会说某站断了是哪一层(收官验收 A)
- [ ] PRD 覆盖率自查:对照 PRD 打钩表自评,让 AI 反审出缺口 + 拿到必做项完成度%(收官验收 B)
- [ ]（可选)AI 助手接通:配好 key 后能查真实藏书回答;没 key 也不崩(Step 3)
- [ ] 能把 Vibe Coding 全流程复盘成一张图,说清自己练了什么(Step 4)

打勾的必做部分(读懂速记 + Step 1 + Step 2)完成,今天的目标就达成了——**你的 C/S 图书馆打包上线了。**

课后任务

任务一(必做):把你自己这套系统的**局域网上线**完整走一遍并记录:打两个 jar、找一台电脑当服务器(或用你自己电脑 + 手机热点模拟),另一台 / 邻座填 IP 连上,截图"两台客户端同时在线 + 并发借最后一本只成一个"。写两句话说明:① 为什么多台客户端同时借书还不会超借?(答案要落到"服务端事务 + `WHERE stock>0` 按影响行数"。)② 你这台服务器的局域网 IP 是多少、你是怎么放行防火墙 9000 的?

任务二(可选 / 挑战,需自己的 key):给 AI 图书助手再加一个工具,比如 `getMyBorrow(userId)`,查某位读者的借阅记录(仍走 Socket 发 `MY_BORROW` 给服务端查库)。这样读者能问"我借了哪些书还没还",助手调这个新工具去查真实数据再回答。参考:

```
给 AI 图书助手再加一个工具 getMyBorrow(userId):走 Socket 发 MY_BORROW 给服务端查借阅记录;
把它也加进 Function Calling 的工具列表,让大模型遇到"我借了什么书"时调用它。
只加这个新工具,别动 searchBooks 和其他逻辑。
```

做完你会体会到:**加一个能力 = 加一个工具描述 + 一个查库实现**,大模型自己会判断什么时候用哪个。这就是 AI 应用"可扩展"的核心套路。

> 记住今天最重要的一句话:**代码写得再好,跑不起来给人用就等于零。** 打包上线,走完最后一公里,系统才真正"活"了。